Tomasulo Multi-Issue Assignment Report

Author: Ahmed Ahmed

Rerun: October 15, 2026, 6:38 PM CDT | Python 3.12.14 on Linux

1. Brief description what you have added to the code

I started with the supplied Python simulator and extended it to support issue widths of 1, 2, 3, and 4. Instructions still issue in program order. In each cycle, the simulator issues up to the selected width and stops when the next instruction cannot get a ROB entry or reservation station. The default issue width is 1.

Addition	What I changed
Multi-issue and renaming	I issue instructions sequentially within each group and capture source operands before updating the destination tag. This preserves RAW, WAR, and WAW dependencies when several instructions issue together.
Configuration and output	I added parsing for issue width, hardware settings, registers, and memory. The command line can override issue width. Output includes ISSUE, EX, MEM, WB, COMMIT, register values, and nonzero memory.
Branch and memory handling	I added the 8-entry BTB and one-bit prediction, recovery of wrong-path instructions, forwarding from the youngest matching older store, and load replay when an unresolved older store later conflicts.
Code organization and tests	I reorganized the starter into parser, model, engine, and output modules. I added seven benchmarks, an independent sequential interpreter, automated tests, and an experiment runner. The original starter remains available for comparison.

What I kept in the processor model

Only issue bandwidth changes with issue width. There is one CDB and one in-order commit slot. Integer and address units are unpipelined, while FP units are pipelined. A result is usable in the following cycle, and stores update memory only at commit. Branch recovery resumes correct fetch two cycles after detecting a misprediction.

Rerun and validation

I reran the code without changing the simulator. All **44 test methods passed**, including **72 seeded differential configurations**. I then ran seven benchmarks at all four issue widths, giving **28 passing configurations**. The command-line smoke test also passed. The result tables were unchanged from the earlier run.

```
python3 -m unittest discover -s tests -v
python3 run_experiments.py
python3 main.py benchmarks/corrected_starter.in --issue-width 4
```

Input correction and limitations

The supplied loop used branch offset -12, which leaves the program under the required PC+4+(offset<<2) rule. I preserved it and tested a corrected -8 version that repeats the full loop and retires 25 instructions. No known failing cases remain in the included tests. FP uses Python arithmetic, and memory is modeled as 64 aligned words at byte addresses 0-252.

2. Show results of test cases under different issue width

The table shows total simulated cycles. Lower is better. Hardware settings are fixed within each test case; only issue width changes. Retired is the number of committed dynamic instructions. Correctness means that all 64 register values and 64 memory words match the independent sequential interpreter.

Test case	Retired	Width 1	Width 2	Width 3	Width 4	Correctness
Corrected starter loop	25	66	65	65	65	4 / 4 PASS
Independent mixed operations	28	47	48	47	47	4 / 4 PASS
Dependency / renaming chain	14	40	38	38	36	4 / 4 PASS
Store-to-load forwarding	9	44	44	44	44	4 / 4 PASS
Branch recovery	17	44	44	44	44	4 / 4 PASS
Memory disambiguation / replay	9	34	34	35	35	4 / 4 PASS
Small ROB / RS / LSQ	14	42	42	42	42	4 / 4 PASS

Inputs are in benchmarks/. Full stage traces and final machine state are in results/<test>_w<width>.txt and .json; summary.csv records all 28 runs.

What I observed

The dependency test improved from 40 cycles at width 1 to 36 at width 4. That is 10% fewer cycles, or a 1.11x speedup (40 / 36). The corrected starter improved from 66 to 65 cycles. Most other tests changed little because dependencies, the single CDB, memory service, and limited queues still restrict progress.

Wider issue was not always faster. The independent test took 48 cycles at width 2 versus 47 at width 1, and the replay test took 35 cycles at widths 3 and 4 versus 34 at width 1. Issue width changes scheduling and contention, so the improvement is not necessarily monotonic.

Selected final results - identical at widths 1, 2, 3, and 4

Test	Checked result
Corrected starter	R1=0, R2=20; F2=3, F4=9, F6=15, F20=6. Mem[24]=15, Mem[28]=11, Mem[32]=7.
Forwarding	F4=7, F5=7; Mem[0]=4, Mem[4]=7, Mem[8]=11. Two load-forwarding events occurred in each run.
Branch recovery	R3=108, R4=112, R5=113; 17 instructions retired. Wrong-path store was discarded; Mem[0] remained 10.
Memory replay	One replay occurred per run. F2=9, F3=18; Mem[16]=9 and Mem[20]=18.

Evidence: results/test_log.txt, experiment_log.txt, cli_smoke_log.txt, validation.json, and summary.csv. Tested simulator source: commit 739dabd944cd.